Tree Borrows

Neven Villani,[1] Johannes Hostert,[2] Derek Dreyer,[3] Ralf Jung[2]

PLDI'25

2025-06-19

[1]Univ. Grenoble Alpes, Verimag
[2]ETH Zurich
[3]MPI-SWS

Rust's type system provides powerful optimizations

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
  *x = 13;
  *y = 20;
  *x
}
```

Rust's type system provides powerful optimizations

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
  *x = 13;
  *y = 20;
  *x
}
```

mutable thus disjoint

*x is unchanged

Rust's type system provides powerful optimizations

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
  *x = 13;
  *y = 20;
  *x
}
```

mutable thus disjoint

*x has known value

*x is unchanged

always returns 13

Rust's type system provides powerful optimizations

```
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
  *x = 13;
  *y = 20;
  13 // formerly *x: one fewer load from memory
}
```

Type-level guarantees for references

~~aliasing~~
~~&~~
~~mutability~~

`&mut` → mutation, no aliasing

`&` → aliasing, no mutation

Escape hatch: `unsafe`

Can **bypass typechecks** to implement **low-level manipulations**

```
unsafe {
  // Code within this block has relaxed typechecking
  ...
}
```

Within `unsafe` it is **the programmer's responsibility** to check

- that pointers are non-null
- that memory is initialized
- …

What if `unsafe` code is misused ?

```rust
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
  *x = 13;
  *y = 20;
  *x
}

fn main() {
  let mut root = 42;
  let ptr = &raw mut root;
  let x = unsafe { &mut *ptr };
  let y = unsafe { &mut *ptr };
  println!("{}", write_both(x, y));
}
```

What if unsafe code is misused ?

```rust
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
  *x = 13;
  *y = 20;
  *x
}

fn main() {
  let mut root = 42;
  let ptr = &raw mut root;
  let x = unsafe { &mut *ptr };
  let y = unsafe { &mut *ptr };
  println!("{}", write_both(x, y));
}
```

What if unsafe code is misused ?

```rust
fn write_both(x: &mut i32, y: &mut i32) -> i32 {
  *x = 13;
  *y = 20;
  *x
}

fn main() {
  let mut root = 42;
  let ptr = &raw mut root;
  let x = unsafe { &mut *ptr };
  let y = unsafe { &mut *ptr };
  println!("{}", write_both(x, y));
}
```

Not the compiler's responsibility

Within `unsafe` it is **the programmer's responsibility** to check

- that pointers are non-null
- that memory is initialized
- compliance with aliasing rules NEW!

Tree Borrows (TB): defines those aliasing rules

Not the compiler's responsibility

Within `unsafe` it is **the programmer's responsibility** to check

- that pointers are non-null
- that memory is initialized

Sounds familiar?

Stacked Borrows has the same purpose,

Tree Borrows is its successor.

Stacked Borrows (SB)

In safe Rust, the Borrow Checker makes borrows well-bracketed.
Stacked Borrows extends the well-bracketedness to unsafe.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.
Stacked Borrows extends the well-bracketedness to `unsafe`.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

root

- new stack at `root`

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.
Stacked Borrows extends the well-bracketedness to unsafe.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

root

- ✓ root is at the top

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.

Stacked Borrows extends the well-bracketedness to unsafe.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

ptr
root

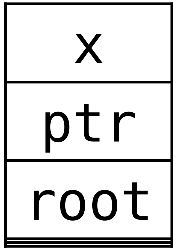
- ✓ root is at the top
- push ptr

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.

Stacked Borrows extends the well-bracketedness to `unsafe`.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```



- ✓ `ptr` is at the top
- push `x`

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.
Stacked Borrows extends the well-bracketedness to unsafe.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

x
ptr
root

- pop until `ptr` is at the top

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.

Stacked Borrows extends the well-bracketedness to `unsafe`.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

ptr
root

- pop until `ptr` is at the top

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.

Stacked Borrows extends the well-bracketedness to `unsafe`.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

y
ptr
root

- pop until `ptr` is at the top
- push y

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.
Stacked Borrows extends the well-bracketedness to `unsafe`.

```rust
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

y
ptr
root

- search for `x`

Desired outcome: UB

In safe Rust, the Borrow Checker makes borrows well-bracketed.

Stacked Borrows extends the well-bracketedness to unsafe.

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
```

UB!

y
ptr
root

Can't use x if it is not in the stack

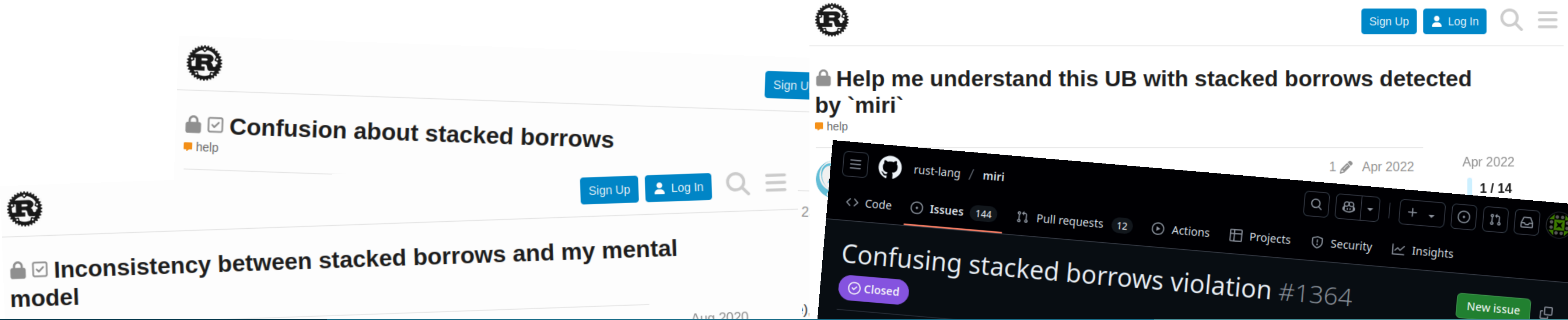
Desired outcome: UB

SB was **implemented** in Miri (official interpreter and UB detector)
$\rightarrow$ included in many projects' CI
$\rightarrow$ several bugs detected (e.g. in stdlib)

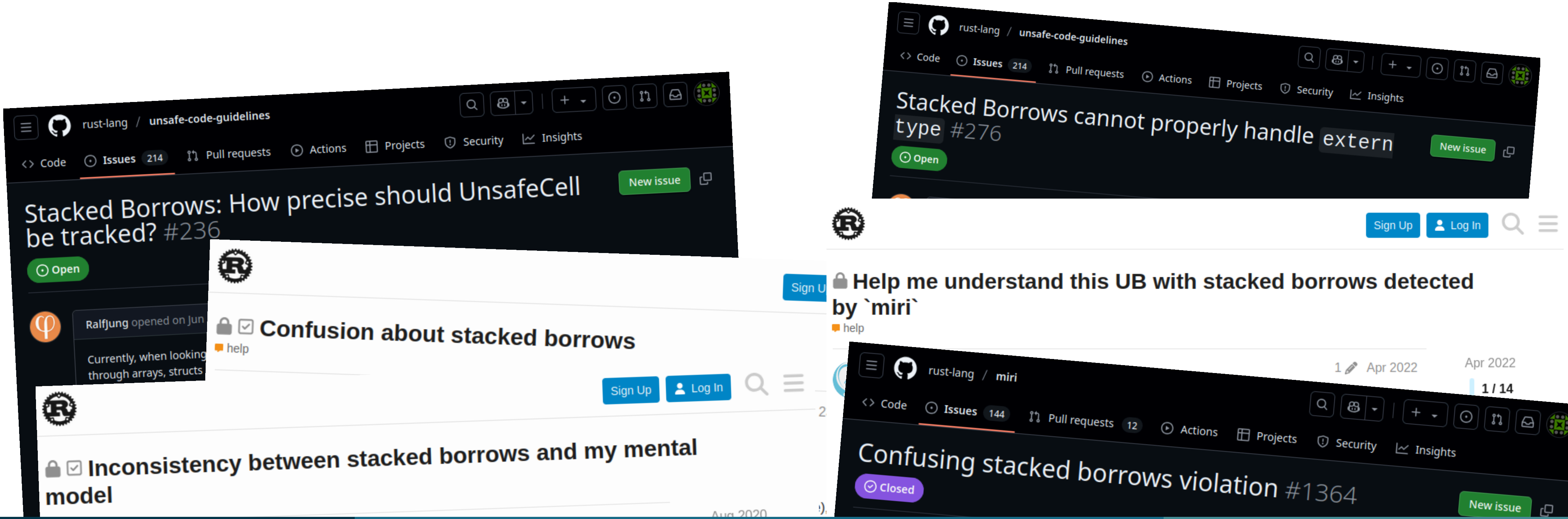
SB was **implemented** in Miri (official interpreter and UB detector)

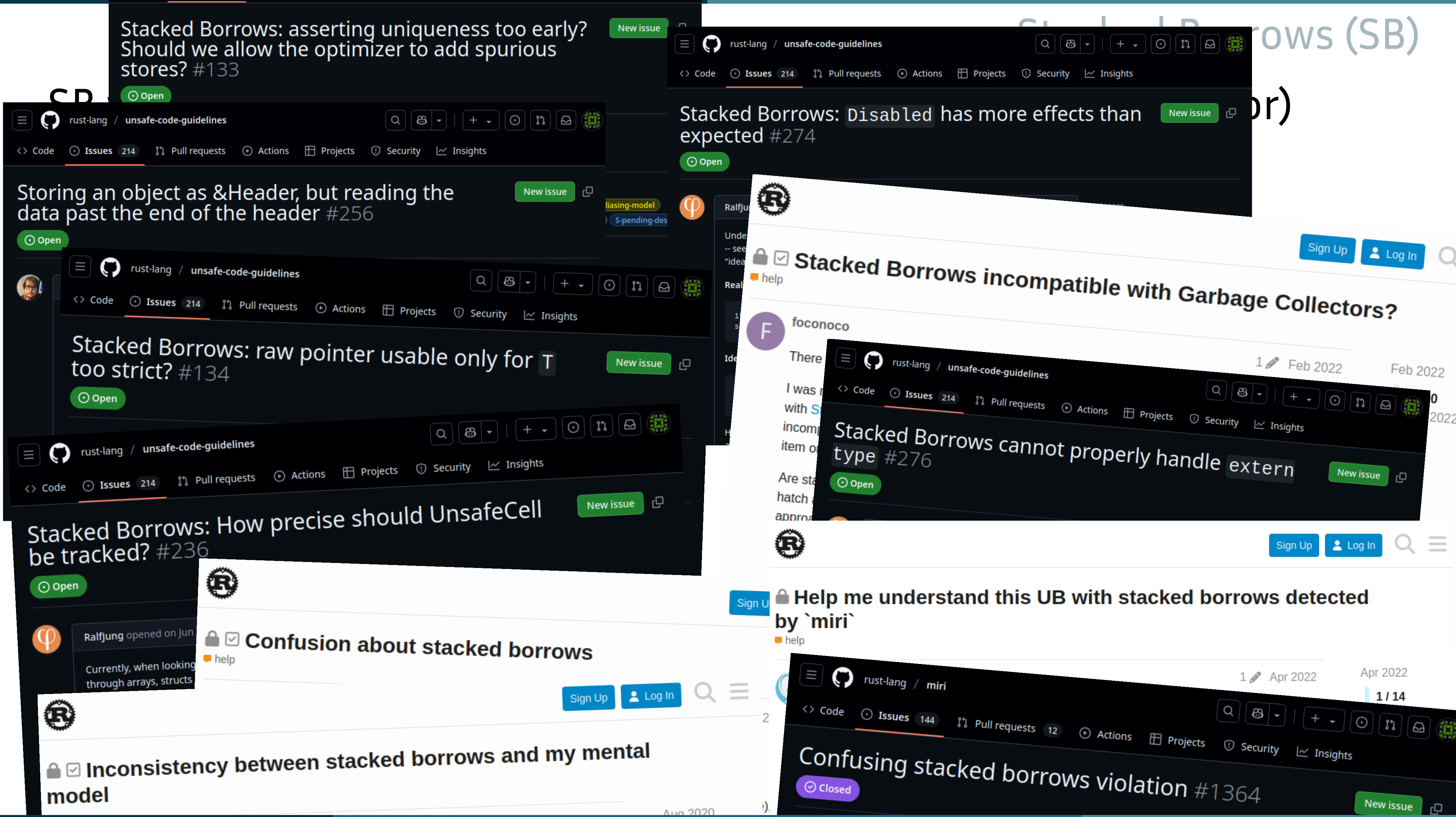
$\rightarrow$ included in many projects' CI

$\rightarrow$ several bugs detected (e.g. in stdlib)



SB was **implemented** in Miri (official interpreter and UB detector)
$\longrightarrow$ included in many projects' CI
$\longrightarrow$ several bugs detected (e.g. in stdlib)

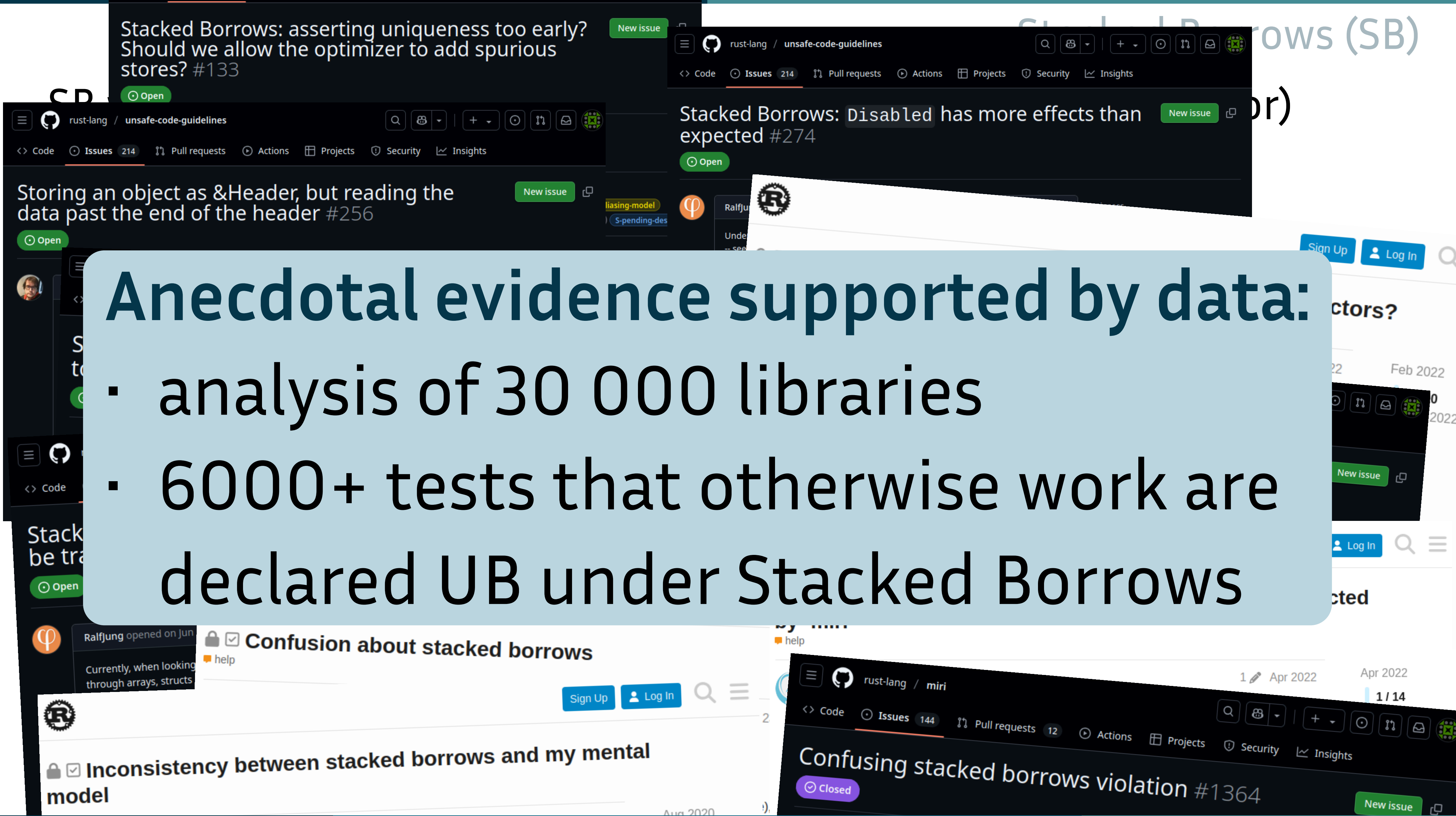




Stacked Borrows: asserting uniqueness too early? Should we allow the optimizer to add spurious stores? #133
Open
rust-lang / unsafe-code-guidelines
Stacked Borrows: Disabled has more effects than expected #274
Open
Storing an object as &Header, but reading the data past the end of the header #256
Open
Stacked Borrows: raw pointer usable only for T too strict? #134
Open
Stacked Borrows incompatible with Garbage Collectors?
help
foconoco
Stacked Borrows cannot properly handle extern type #276
Open
Stacked Borrows: How precise should UnsafeCell be tracked? #236
Open
Help me understand this UB with stacked borrows detected by `miri`
help
Confusion about stacked borrows
help
rust-lang / miri
Confusing stacked borrows violation #1364
Closed
Inconsistency between stacked borrows and my mental model

Anecdotal evidence supported by data:

- analysis of 30 000 libraries
- 6000+ tests that otherwise work are declared UB under Stacked Borrows



Tree Borrows allows much more code

Tree Borrows uses a **tree** instead of a stack to track borrows

Out of 30 000 most downloaded libraries,
$> 50\%$ **fewer** tests with aliasing UB when using Tree Borrows

Tree Borrows allows much more code

Tree Borrows uses a **tree** instead of a stack to track borrows

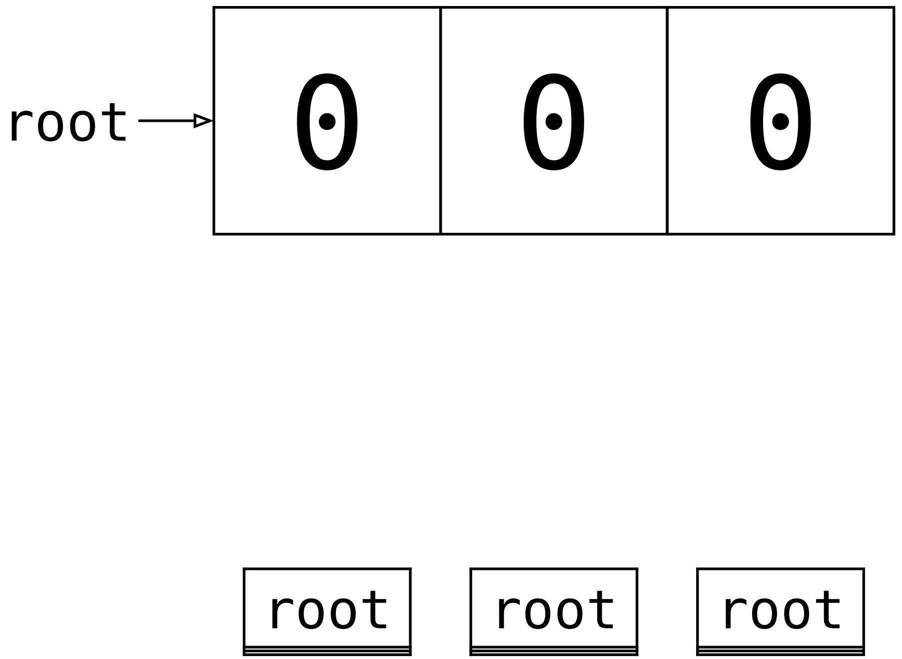
Out of 30 000 most downloaded libraries,
$> \mathbf{50\%}$ **fewer** tests with aliasing UB when using Tree Borrows

fixes known technical limitations of SB, incl. **pointer offsets**

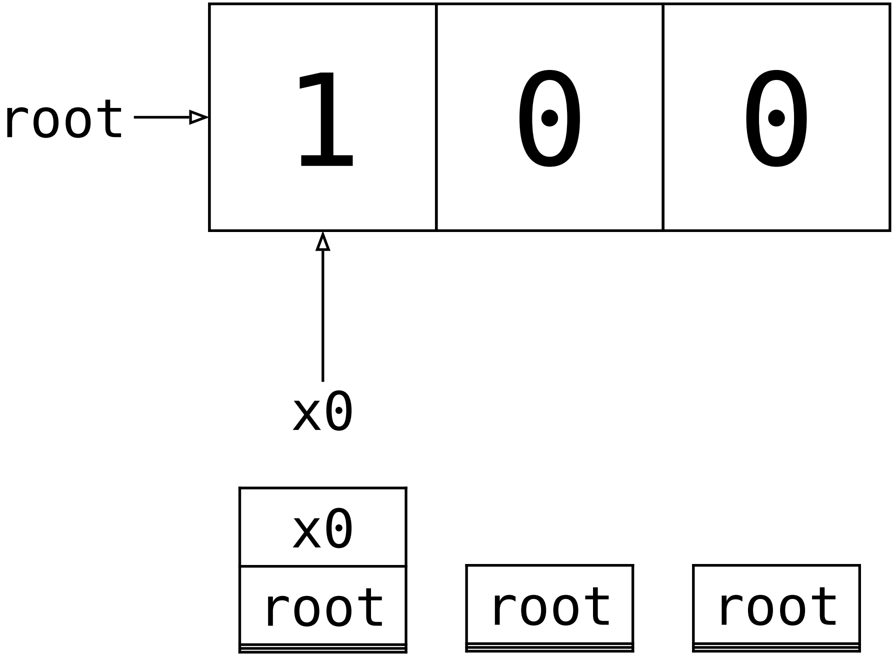
From Stacks to Trees

```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };
```

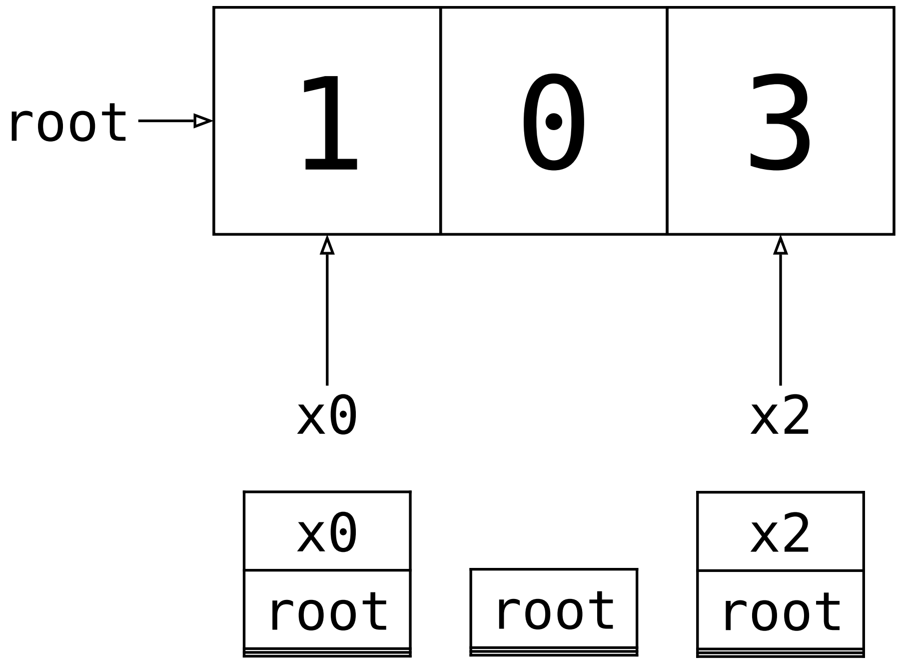
```rust
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };
```



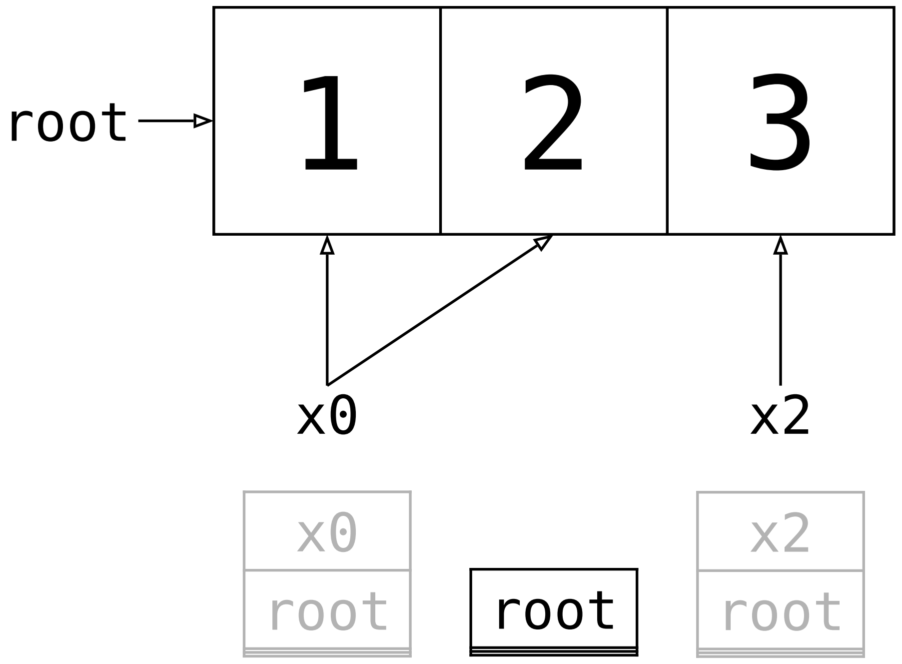
```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };
```



```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };
```

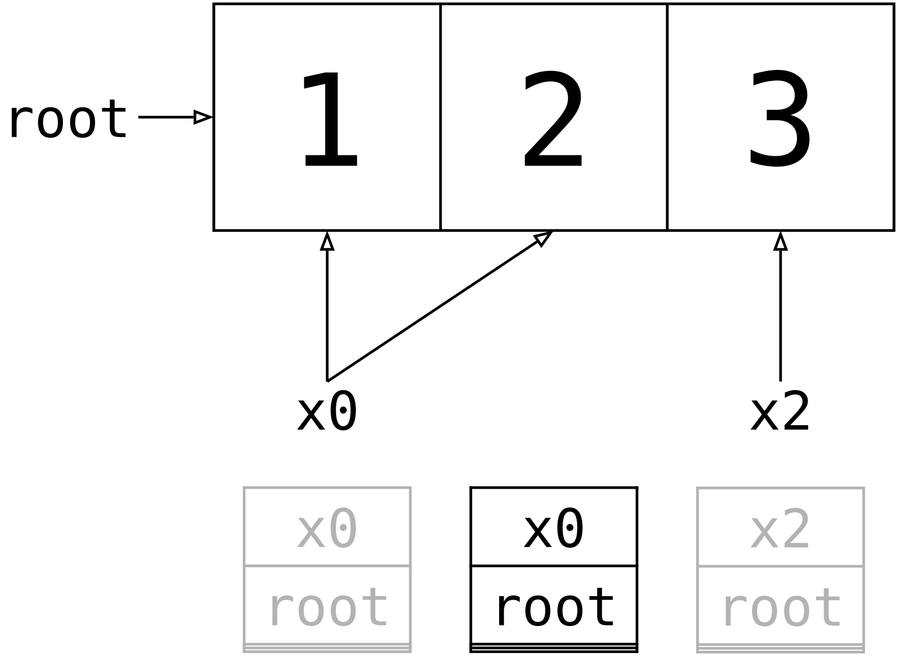


```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };

// Scenario 1
unsafe { *x0.add(1) = 2; }
```



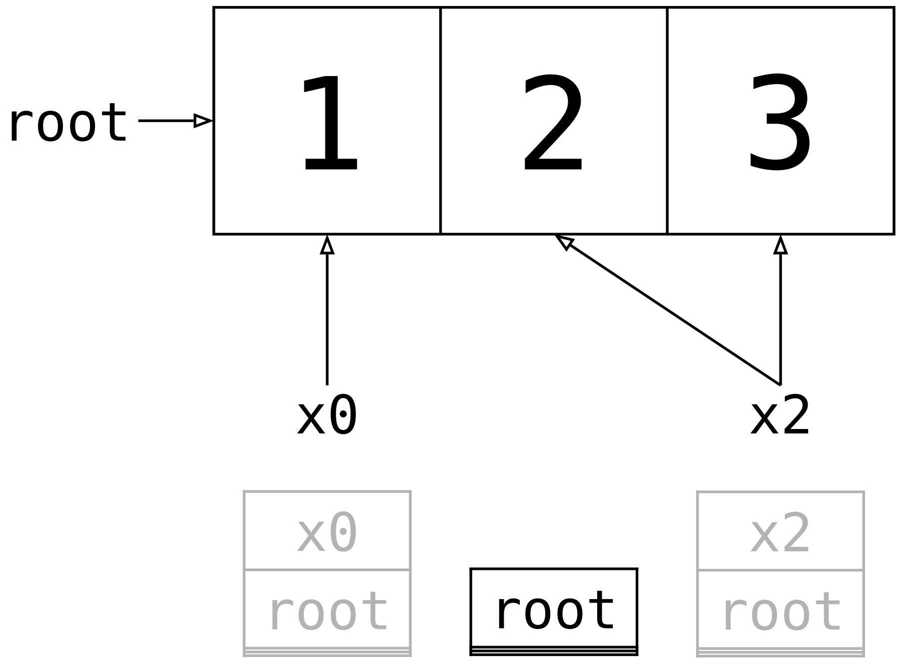
Desired outcome: not UB

```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };

// Scenario 1
unsafe { *x0.add(1) = 2; }
```



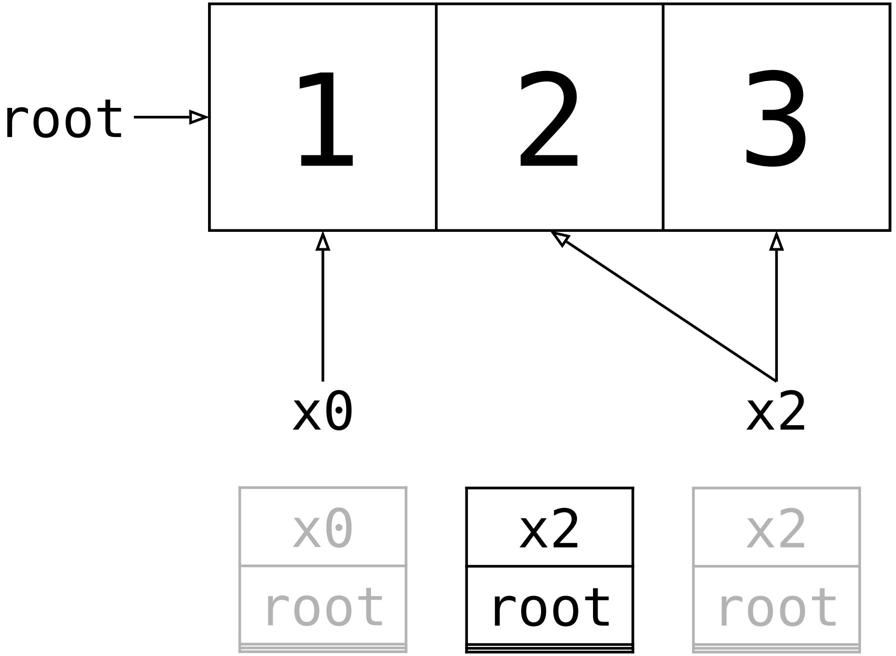
Desired outcome: not UB

```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };

// Scenario 2
unsafe { *x2.sub(1) = 2; }
```



Desired outcome: not UB

```rust
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };

// Scenario 2
unsafe { *x2.sub(1) = 2; }
```

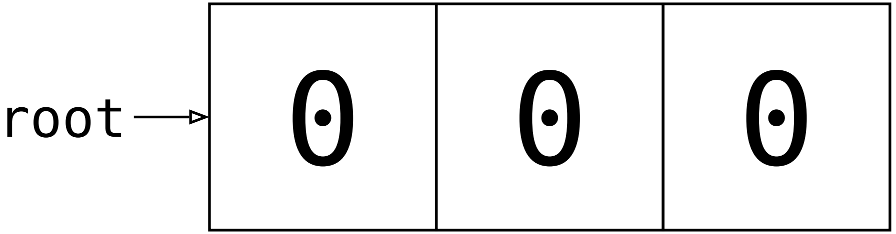


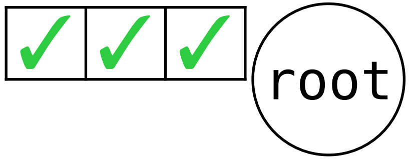
Desired outcome: not UB

```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };
```

Desired outcome: not UB

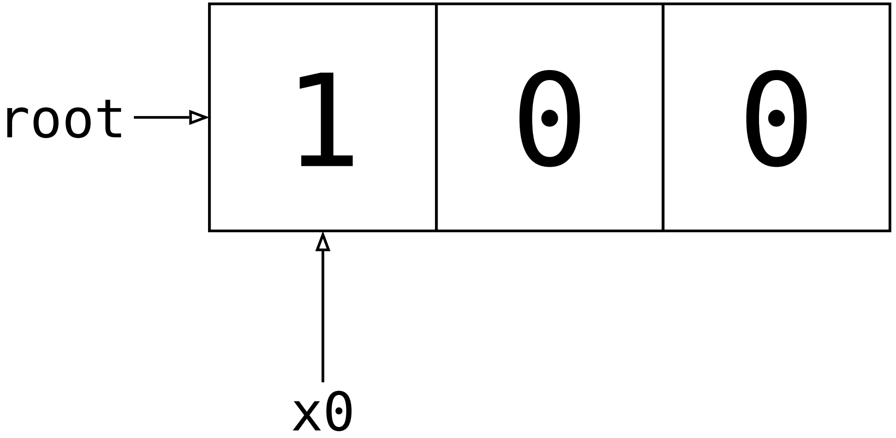
```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };
```

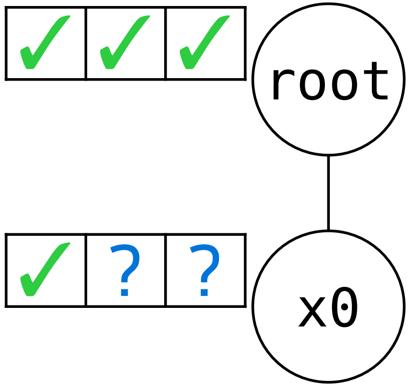




Desired outcome: not UB

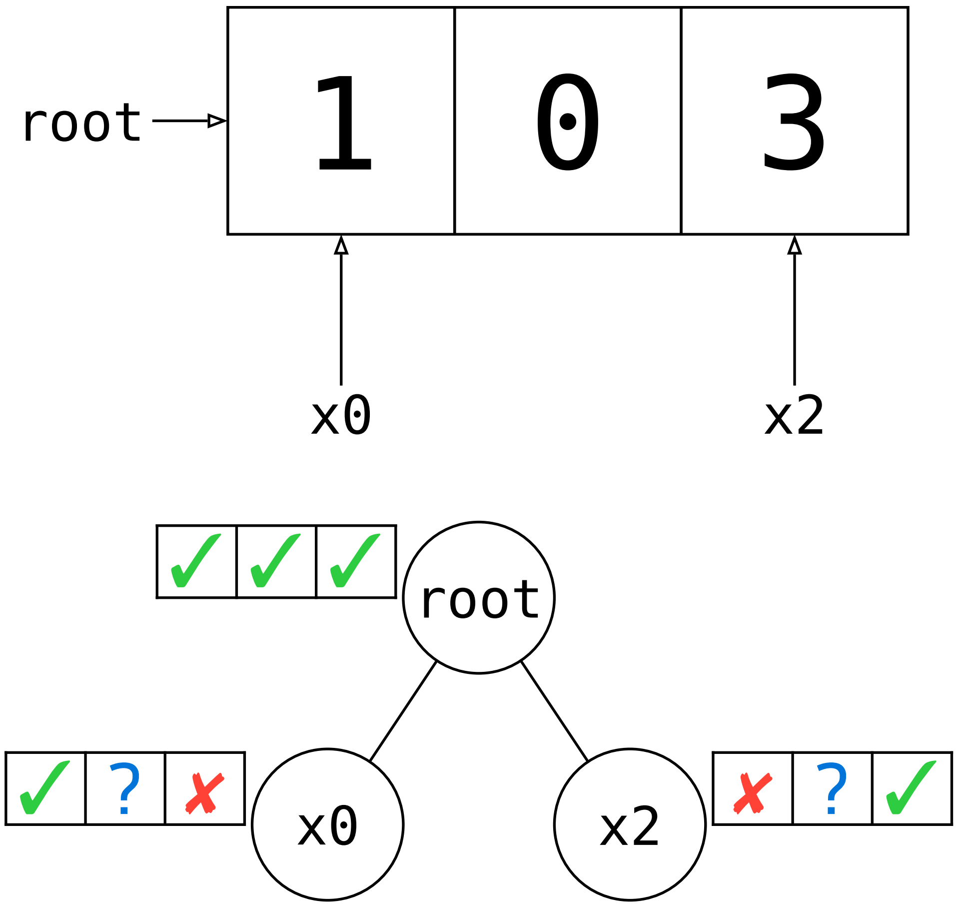
```rust
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };
```





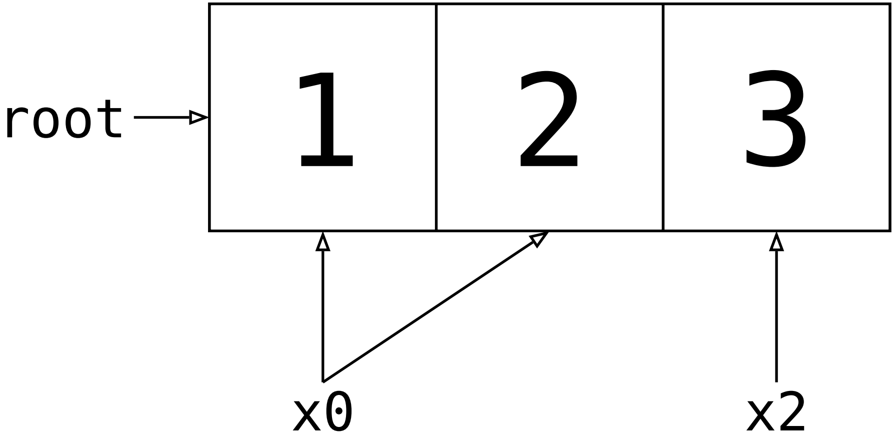
Desired outcome: not UB

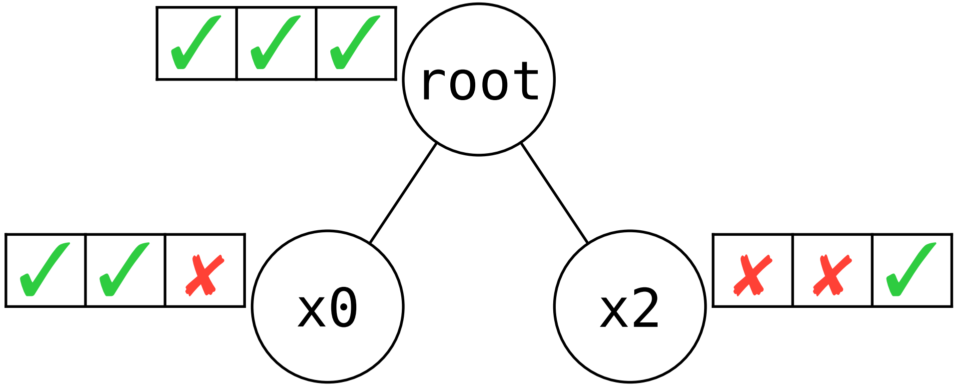
```rust
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };
```



Desired outcome: not UB

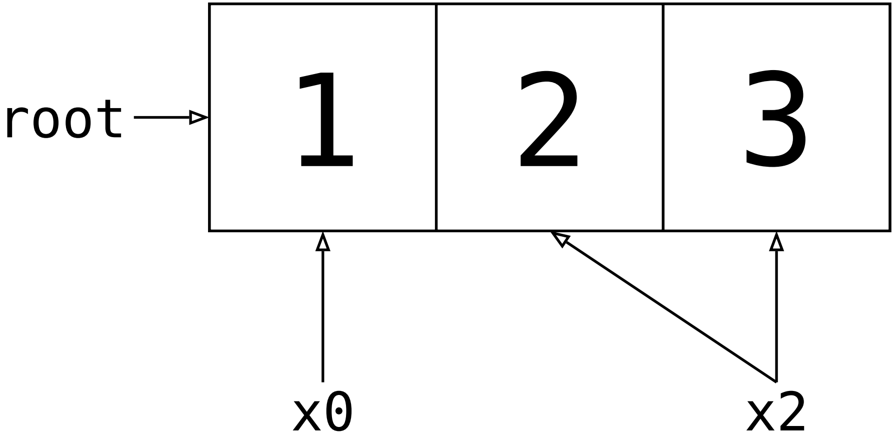
```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };

// Scenario 1
unsafe { *x0.add(1) = 2 };
```

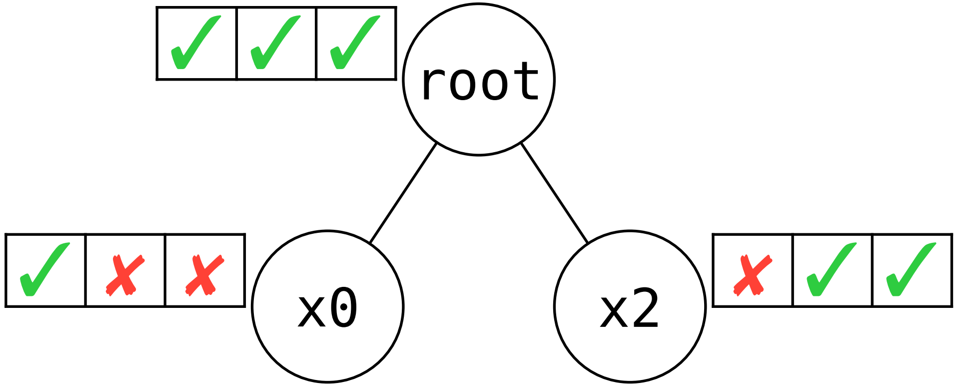




Desired outcome: not UB

```
let mut root = vec![0, 0, 0];
let x0 = &raw mut root[0];
let x2 = &raw mut root[2];
unsafe { *x0 = 1 };
unsafe { *x2 = 3 };

// Scenario 2
unsafe { *x2.sub(1) = 2; }
```





Desired outcome: not UB

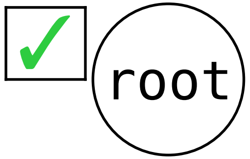
A second look at the motivating example


```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
*y = 20;
```

Desired outcome: UB

A second look at the motivating example

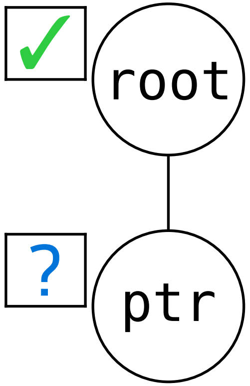

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
*y = 20;
```



Desired outcome: UB

A second look at the motivating example

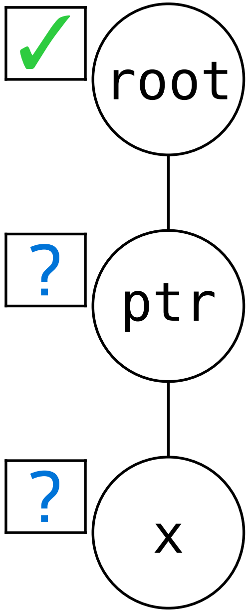

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
*y = 20;
```



Desired outcome: UB

A second look at the motivating example

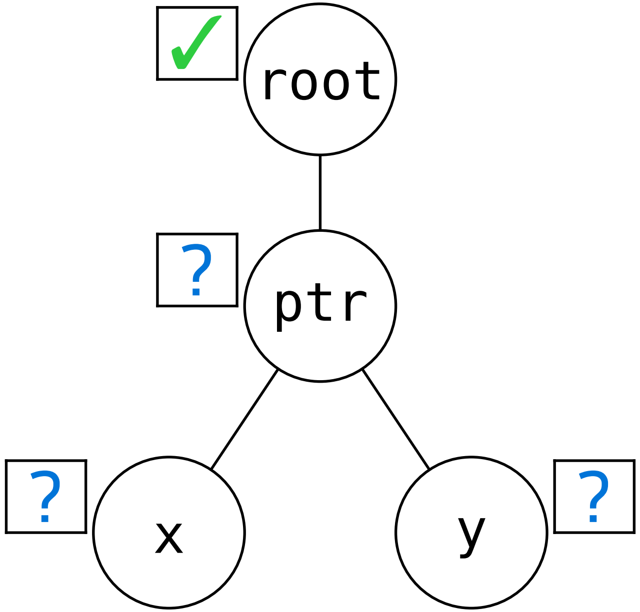

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
*y = 20;
```



Desired outcome: UB

A second look at the motivating example

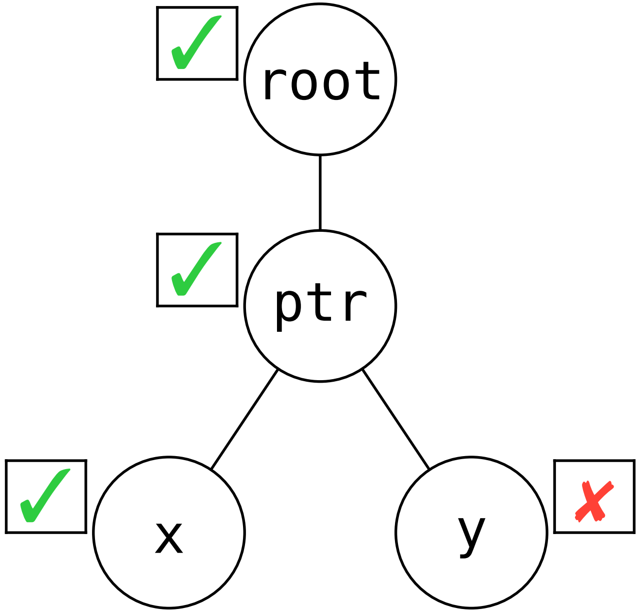

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
*y = 20;
```



Desired outcome: UB

A second look at the motivating example


```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
*y = 20;
```

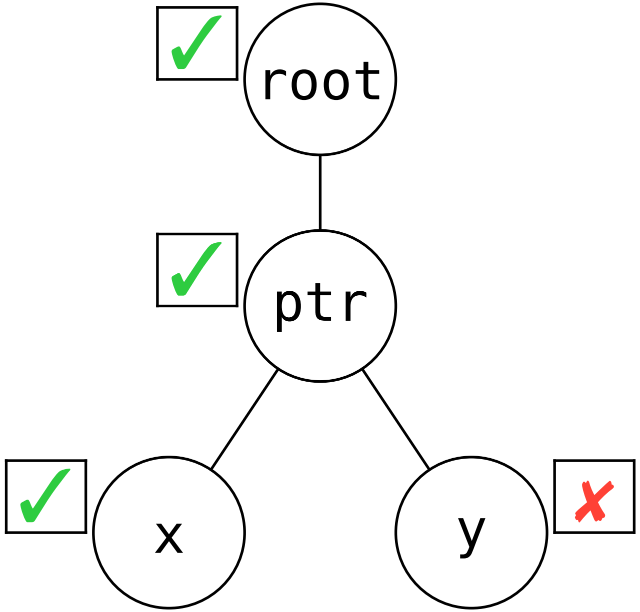


Desired outcome: UB

A second look at the motivating example

```
let mut root = 42;
let ptr = &raw mut root;
let x = unsafe { &mut *ptr };
let y = unsafe { &mut *ptr };
// inline write_both(x, y):
*x = 13;
*y = 20;
```

UB!



Desired outcome: UB

Evaluation

TB should enable desired optimizations

i.e. have enough UB to rule out problematic patterns

- formalized in Rocq (+Simuliris)
- a selection of optimizations proven
  - ✓ delete read through `&mut` or `&`
  - ✓ insert read through `&` in function
  - ✓ move read down for `&mut` or `&` in function

  ...

It should be possible to write unsafe code free of UB

i.e. UB should be predictable and not too common

Tree Borrows reduces aliasing-related UB by over 50%

Only 31 ($< 0.01\%$) tests are regressions, all easily fixable.

(Out of 30 000 libraries, 400 000+ working tests)

“Tree Borrows accepts more real-world programs that call foreign functions than Stacked Borrows due to differences in handling pointer arithmetic.”

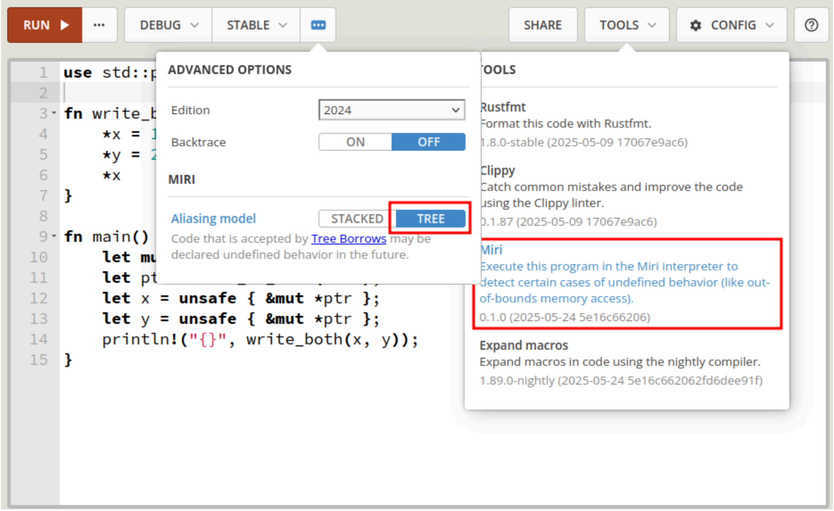
A Study of Undefined Behavior Across Foreign Function Boundaries in Rust Libraries, by I. McCormack, J. Sunshine, J. Aldrich @ ICSE’25

Conclusion

Try it out:

Rust Playground supports TB

`play.rust-lang.org`



Learn more:

`plf.inf.ethz.ch/research/pldi25-tree-borrows.html`

- detailed state machine
- raw pointers
- interior mutability